



Linguagem de Programação

Professor: Euclides Paim
euclidespaim@gmail.com



Gerenciamento de Pacotes

PIP e VENV

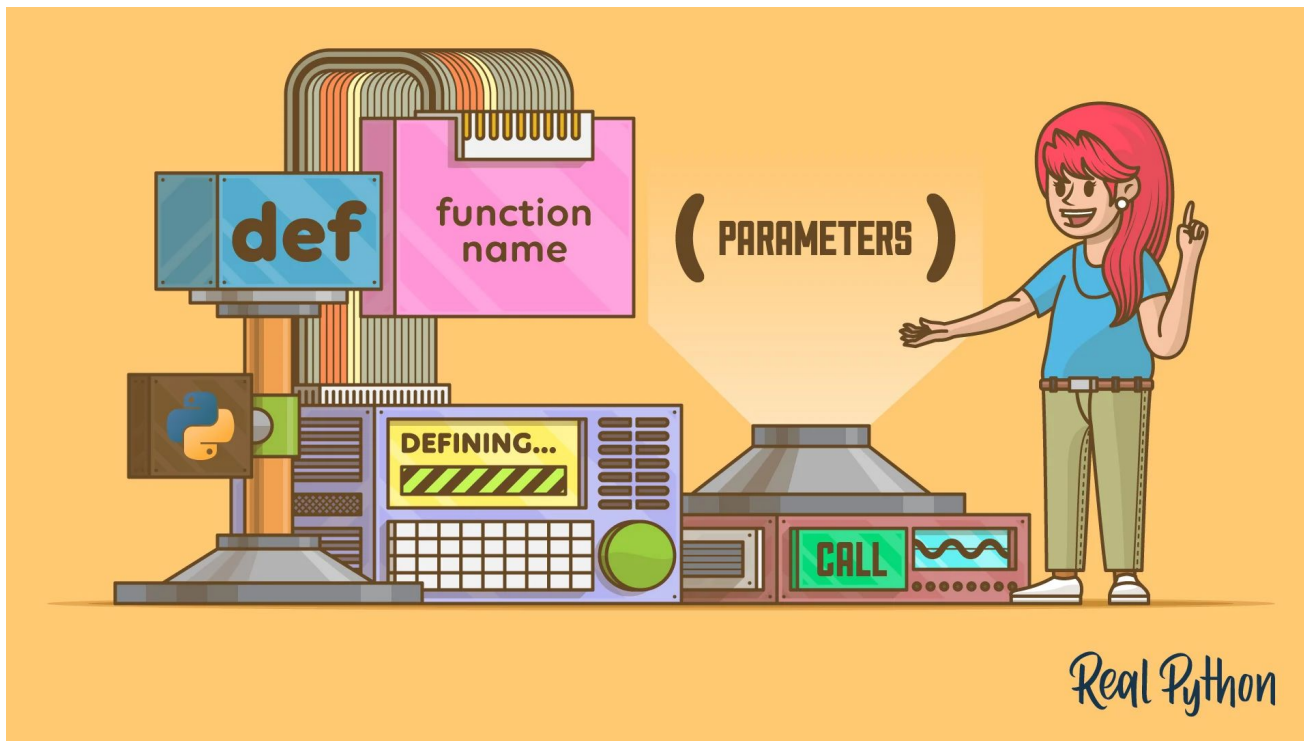
Professor: Euclides Paim

euclidespaim@gmail.com



Linguagem de Programação

Gerenciamento de Pacotes





Linguagem de Programação

Gerenciamento de Pacotes

Sumário

- O que é o PIP?
- Verificando a instalação do PIP
- O desafio das permissões (Active Directory)
- Ambientes Virtuais (venv)
- Comandos básicos do PIP
- Exemplo Prático de Código
- Boas práticas: requirements.txt





Linguagem de Programação

Gerenciamento de Pacotes

O que é o PIP?

Definição Teórica: O PIP (*Pip Installs Packages*) é o gerenciador de pacotes oficial da linguagem Python. Ele permite instalar, atualizar e remover bibliotecas ou módulos de terceiros que não fazem parte dos módulos nativos (*built-in*).

Analogia: Se os módulos nativos são os aplicativos que já vêm de fábrica no seu celular, o PIP é a "App Store" ou a "Play Store". Ele conecta você a um repositório mundial (o *PyPI - Python Package Index*) onde milhares de programadores disponibilizam módulos para você usar no seu código.





Linguagem de Programação

Gerenciamento de Pacotes

Definição Teórica:

Nas versões modernas do Python (acima da 3.4), o PIP já vem embutido na instalação padrão. Antes de instalar qualquer coisa, precisamos verificar se ele está acessível pelo terminal.

- **Exemplo de Sintaxe (no Terminal / Prompt de Comando):**

```
# Verifica se o pip está instalado e mostra a versão
python -m pip --version
# ou
pip --version
```

Nota: Se o comando não for reconhecido, o Python pode ter sido instalado sem marcar a opção "Add Python to PATH" ou o PIP não foi selecionado na instalação. Se for o caso, podemos baixar o script `get-pip.py` e rodar o comando `python get-pip.py --user`.



Linguagem de Programação

Gerenciamento de Pacotes

O Desafio das Permissões (Active Directory)

Nos computadores do laboratório da escola, vocês utilizam contas de usuário controladas pelo AD (Active Directory). Isso significa que vocês **não têm** permissões de Administrador para instalar pacotes na pasta raiz do sistema (como `C:\Program Files\Python`).

Se tentarmos rodar um `pip install <pacote>` globalmente, o sistema operacional irá bloquear.

A Solução: Para contornar isso, vamos utilizar os **Ambientes Virtuais (venv)** ou a flag de usuário local (`--user`)

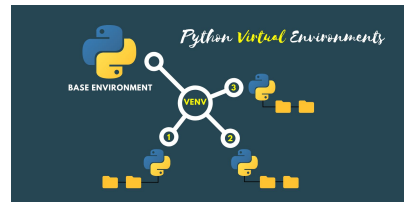


Linguagem de Programação

Gerenciamento de Pacotes

Ambientes Virtuais (venv)

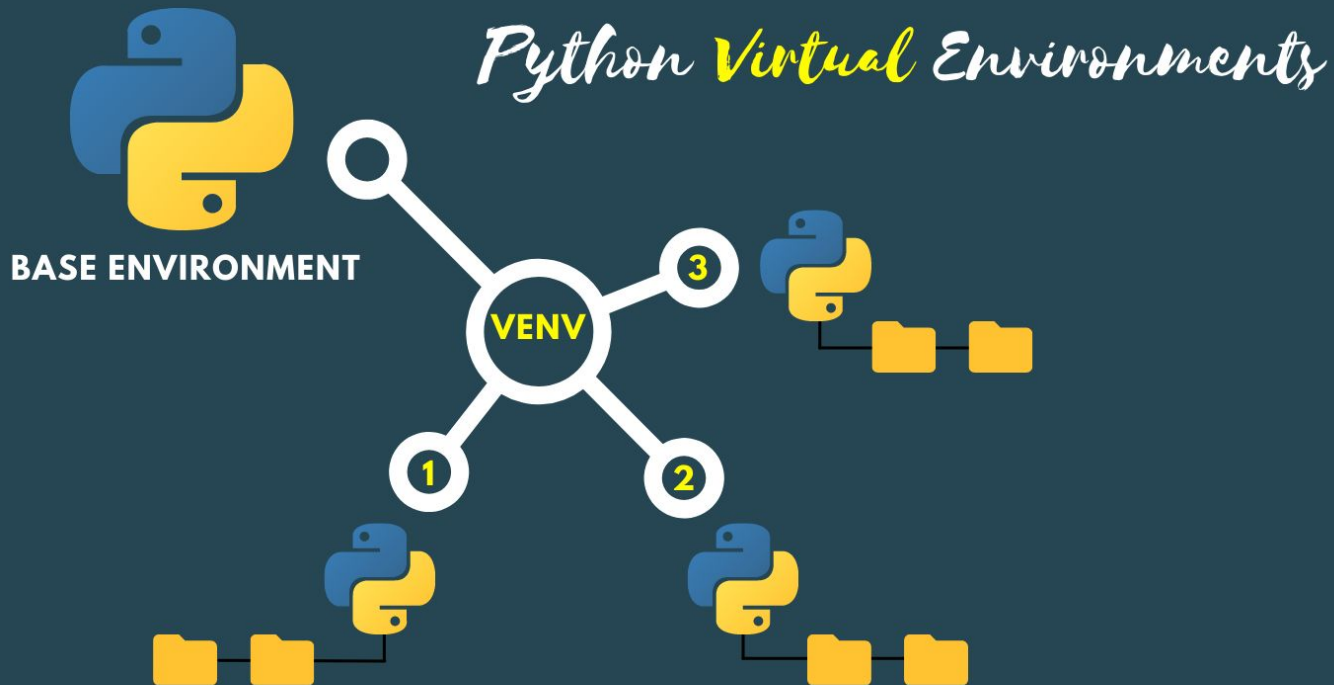
- **Definição Teórica:** Um ambiente virtual (**venv**) é uma pasta autônoma dentro do seu projeto que contém sua própria cópia do executável do Python e do PIP. Tudo que você instala dentro dele fica isolado ali. Como a pasta do projeto pertence ao seu usuário logado, o AD **não bloqueia** a instalação.
- **Analogia:** É como criar uma "bolha" isolada para o seu projeto. O que acontece na bolha, fica na bolha. Isso evita que os módulos de um projeto entrem em conflito com os de outro, e burla a necessidade de ser administrador da máquina inteira.





Linguagem de Programação

Gerenciamento de Pacotes





Linguagem de Programação

Gerenciamento de Pacotes

Criando e Ativando um venv

- Exemplo de Sintaxe (no Terminal da sua pasta do projeto):

```
# 1. Criar o ambiente virtual (cria uma pasta chamada 'meu_env')
python -m venv meu_env

# 2. Ativar o ambiente virtual (No Windows)
meu_env\Scripts\activate

# 3. Ativar o ambiente virtual (No Linux, caso usem)
source meu_env/bin/activate
```

Nota: Para desativar o **venv** basta digitar: **deactivate**



Linguagem de Programação

Gerenciamento de Pacotes

Exemplo Prático: Módulo Colorama

Vamos usar o módulo de terceiros `colorama`, que permite imprimir textos coloridos no terminal.

- 1º Passo no terminal: `pip install colorama`
- 2º Passo no arquivo `main.py`:

```
# Importando as funções do módulo instalado
from colorama import Fore, Back, Style, init

# Inicializa o colorama (necessário no Windows)
init()

# Usando as variáveis do módulo para colorir o texto
print(Fore.GREEN + "Olá, Turma! Este texto é verde.")
print(Back.RED + Fore.WHITE + "Fundo vermelho e texto branco!")
print(Style.RESET_ALL + "Texto normal novamente.")
```



Linguagem de Programação

Gerenciamento de Pacotes

Módulos Populares: Buscando dados na Web (`requests`)

Definição Teórica: O módulo `requests` permite que o seu código Python acesse a internet, buscando informações de outros sites e serviços (APIs) de forma muito simples. Ele é a ponte entre o seu script local e o mundo online. **Como instalar no venv:**
`pip install requests`

Exemplo de Sintaxe (Buscando um Endereço):

```
import requests

# Consultando a API pública e gratuita do ViaCEP
resposta = requests.get('https://viacep.com.br/ws/88301320/json/')
dados = resposta.json() # Transforma o resultado em um dicionário Python

print("--- Dados Encontrados ---")
print(f"Rua: {dados['logradouro']}")
print(f"Bairro: {dados['bairro']}")
print(f"Cidade/Estado: {dados['localidade']} - {dados['uf']}")
```

Dica: Modifiquem o CEP no código para o da casa de vocês ou o da nossa escola aqui na rede estadual e vejam a "mágica" acontecer no terminal.



Linguagem de Programação

Gerenciamento de Pacotes

Módulos Populares: Arte no Terminal (`pyfiglet`)

Definição Teórica: A programação também pode ser criativa dentro do terminal preto e branco! O `pyfiglet` é um módulo clássico que transforma strings comuns em "Arte ASCII" (textos gigantes desenhados com caracteres).

Como instalar no venv: `pip install pyfiglet` - **Exemplo de Sintaxe:**

```
import pyfiglet

# Criando o texto estilizado
texto_arte = pyfiglet.figlet_format("Turma 202 !", font="slant")

print(texto_arte)
```



Linguagem de Programação

Gerenciamento de Pacotes

Módulos Populares: Gerador de QR Code (`qrcode`)

Definição Teórica: Este módulo permite gerar códigos QR (Quick Response) de forma automatizada. É uma excelente ferramenta para criar aplicativos que compartilham links, contatos ou crachás de identificação. **Como instalar no venv:** `pip install`

`qrcode` - **Exemplo de Sintaxe:**

```
import qrcode

# O texto ou link que o QR Code vai armazenar
dados = "https://www.python.org"

# Gerando a imagem
imagem = qrcode.make(dados)

# Salvando a imagem na mesma pasta do seu projeto
imagem.save("meu_site.png")
print("QR Code gerado e salvo como 'meu_site.png'!")
```



Linguagem de Programação

Gerenciamento de Pacotes

Boas Práticas: requirements.txt

- **Definição Teórica:** Quando você envia seu projeto para outra pessoa (ou para o GitHub), você **não envia** a pasta do ambiente virtual (`meu_env`). Em vez disso, você gera um arquivo de texto chamado `requirements.txt` com a lista de dependências.
- **Exemplo de Sintaxe (Terminal):**

```
# Salvar a lista de pacotes no arquivo:
```

```
pip freeze > requirements.txt
```

```
# Para instalar tudo que está no arquivo (no PC de outro aluno):
```

```
pip install -r requirements.txt
```

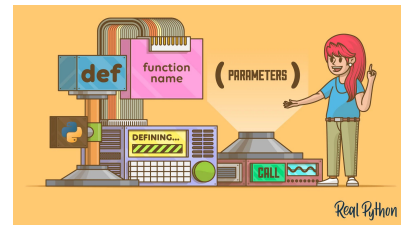


Linguagem de Programação

Gerenciamento de Pacotes

Conclusão e Boas Práticas

- **Regra Importante:** Nunca instale pacotes globalmente se não for estritamente necessário. Trabalhe sempre dentro de um `venv`.
- **Organização:** O isolamento garante que as restrições do sistema escolar (AD) não atrapalhem a criação de aplicativos complexos.





That's all Folks!



Referências

Referências Básicas

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. ***Algoritmos: teoria e prática***. 3. ed. Rio de Janeiro: Elsevier, 2013.

SEBESTA, Robert W. ***Conceitos de linguagens de programação***. 10. ed. São Paulo: Pearson, 2018.

MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. ***Algoritmos: lógica para desenvolvimento de programação de computadores***. 27. ed. São Paulo: Érica, 2016.

DOWNEY, Allen B. ***Pense em Python: como pensar como um cientista da computação***. Tradução de Cássio de Souza Costa. 2. ed. São Paulo: Novatec, 2016.

Referências Complementares

IEPSEN, Edécio Fernando. ***Lógica de programação e algoritmos com JavaScript***. 2. ed. São Paulo: Novatec, 2022.

Referências na Internet

<https://www.w3schools.com/python/>